

Hands On Module 1

Artificial Intelligence

Hands-On Task

Task Overview	You will perform a complete exploratory data analysis (EDA) on a Spotify music dataset using NumPy and Pandas, while integrating AI coding tools as part of your workflow. The dataset contains audio features for tens of thousands of songs across multiple genres. You will answer specific analytical questions, build a clean data pipeline, and document your findings.
Dataset	Spotify Songs Dataset (TidyTuesday 2020): url = "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2020/2020-01-21/spotify_songs.csv"
Tools Required	Python (Jupyter Notebook / Google Colab), Pandas, NumPy, Gemini Code Assist, Antigravity, Claude Code, GitHub Copilot, Cursor.
Expected Output	A Jupyter Notebook (.ipynb) containing runnable code, visible outputs, and narrative markdown cells explaining every analytical decision. The notebook must run topto-bottom without errors.
Portfolio Alignment	This task builds the structured EDA habit that will serve as the first step in every ML project and your final portfolio project.
Assessment Details	
Assessed on: correctness of code and outputs, analytical quality (are the insights meaningful, or just function calls?), notebook structure and readability; quality of markdown documentation.	

Task Specification

Stage 1: Setup and Initial Inspection (required)

1. Load the dataset directly from the URL above using `pd.read_csv()`.

2. Perform initial inspection: display the shape, dtypes, and the number of missing values per column. If any column has more than 20% missing values, drop it and provide a written justification.
3. Identify and handle duplicate rows. Justify your deduplication strategy, which column(s) did you use, and why?
4. Build a summary table showing, for every numeric column, the mean, median, standard deviation, and interquartile range (IQR). Compute IQR manually using NumPy (no scipy allowed).

Stage 2 Genre and Popularity Analysis (required)

5. Use groupby to compute the distribution statistics (mean, std, median) of track_popularity, danceability, energy, and valence per playlist_genre.
6. Answer: which genre has the highest variance in track popularity? Interpret what this means from a content recommendation business perspective.
7. Find the 10 artists with the most tracks. Among those 10, who has the highest mean track_popularity? Does volume correlate with quality in this dataset?
8. Filter tracks that satisfy all of the following conditions: track_popularity > 70, danceability > 0.7, energy > 0.6, duration_ms < 240000. How many tracks pass? Which genre dominates?

Stage 3 NumPy Analysis (required)

9. Extract the nine numeric audio feature columns (danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo) as a NumPy array. Normalise every feature to the range [0, 1] using min-max scaling vectorised, no Python loops, no sklearn.
10. Compute the correlation matrix of the nine audio features manually using np.corrcoef(). Identify the pair with the highest positive correlation and the pair with the most negative correlation. Interpret both results in musical terms.
11. Use NumPy boolean masking (not Pandas filtering) to identify tracks whose energy value lies above the mean (μ) plus one standard deviation (σ). Compare the mean track_popularity of this subset with the overall mean.

Stage 4 Documentation and AI Tool Reflection (required)

12. Use GitHub Copilot or Gemini Code Assist to help write proper docstrings for at least two functions you wrote in the previous stages. In a markdown cell, paste the prompt you used and evaluate the AI output, did you use it as-is, modify it, or discard it? Why?

13. At the end of the notebook, write a markdown cell containing three key insights from the data, phrased so a non-technical person can understand them.

Bonus 1 – Artist Audio Fingerprint

Description: Write a function `artist_fingerprint(df, artist_name)` that takes the DataFrame and an artist name, and returns the mean vector of that artist's nine audio features as a NumPy array. Then use this function to compare two artists of your choice using cosine similarity implemented from its mathematical definition, no sklearn or scipy:

$$\text{cosine_similarity}(A, B) = A \cdot B / \|A\| \cdot \|B\|$$

Interpret the result: do two artists from the same genre have a higher cosine similarity than artists from different genres? Test at least three artist pairs and discuss the findings.

Bonus 2 – Genre Cluster Prole

Description: Build an analysis that constructs an audio profile for each genre using the following approach:

1. Compute the centroid of each genre (the mean feature vector per genre) as a NumPy matrix of shape `(n_genres, 9)`.
2. Compute the pairwise distance matrix between all genres using Euclidean distance, fully vectorised with NumPy broadcasting, no loops. The result should be a `(n_genres, n_genres)` matrix.
3. Identify the two most similar and the two most different genres by audio prole. Does the result match your musical intuition?

4. Write a function `recommend_similar_genre(genre, top_k=3)` that returns the `top_k` most similar genres based on the distance matrix.

Bonus 3 – Reusable Analysis Class with AI Documentation

Description: Refactor your entire analysis into a Python class called `SpotifyAnalyzer` with at least 5 methods representing the main analysis stages.

Requirements:

- `__init__(self, url)` , load and store the DataFrame.
- Separate methods for each major analysis stage.
- Full type hints on all methods.
- Use an AI coding tool (Copilot or Gemini) to generate comprehensive docstrings for every method. Paste the prompts you used and evaluate the output quality.
- Add a method `generate_report(self)` -> dict that returns a dictionary of all key insights, ready to be printed or serialised to JSON. Demonstrate the class by calling `generate_report()` and displaying the output. The code must run from a single Colab cell with no additional setup.